Chapter 3: Database Integration into a Flask App

In this chapter you will put together what you learned in the previous chapter to create a Flask app that executes database operations when users visit specific URLs.

Creating a simple read-only app

To create a quick visualization, put together all the snippets from the previous chapter in the following `app.py`. Only the highlighted lines are new.

app.py

```
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()

@app.route('/')
def index():
    users = User.query.all() ①
    return render_template('index.html', users=users) ②
```

① This retrieves all the entries from the `User` table

② This renders the template and passes the retrieved User objects to the template.

This server only has one route (`/`) and needs a new template. The function is already passing the `users` variable to the template. Create the `home.html` template as follows and see how it uses the passed variable.

templates/home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard</title>
</head>
<body>
    <h1>List of all users</h1>
    <ul>
        <!-- List all entries retrieved from the database and passed to the
template -->
        {% for user in users %}
            <li>{{ user.username }} - {{ user.email }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

With this set up, you have another way of seeing the changes you make to the table, either through the Flask shell or the methods that will be introduced below. Note that the list only refreshes on reload.

Flash messages

Flash messages are Flask's way of sending notifications to the browser to be displayed to the user (e.g., for errors or confirmations). Insert the following HTML snippet wherever you want the notifications to pop up (e.g., above or below the list of users). Read it thoroughly to make sure you understand every line.

```
<!-- Flash messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert alert-{{ category }}">{{ message }}</div>
        {% endfor %}
    {% endif %}
{% endwith %}
```

- `get_flashed_messages` is a function provided by Flask to retrieved queued notifications.
- Set `with_categories` to true to include the type (e.g., error, success) in the messages.
- Iterate through the list of queued messages and create one `<div>` per message.
- These messages can be styled with the assigned css classes like `alert` and `alert-error`.

You can send messages to the client by adding `flash('Sorry, an error occurred', 'error')` on your server where the first string is the message and the second string the type. You will find examples of these in the following sections. Make sure to import the `flash` function from the Flask package at the top of your script!

Basic database operations (CRUD)

The four fundamental database operations as often called by their acronym "CRUD": create, read, update, delete. Below you will find implementations for each of these operations as different routes in the Flask application.

General explanations:

- Routes containing strings like `<int:user_id>` or `<string:email>` allow you to add parameters to your route. The format is `<datatype:variable_name>`.
- `db.session` is the object that manages all the changes you make to the database. The changes are only committed when you run `db.session.commit()` and can be reverted with `db.session.rollback()` if an error occurred.
- End every route with `return redirect(url_for('index'))` to bring visitors back to the home page, even though they clicked on a link (e.g., to `/add_user/`).

> **URL parameters are not the best method of passing variables and you will learn the correct way in the next chapter.**

CREATE - Add a new user

```
@app.route('/add_user/<string:username>/<string:email>') ❶
def add_user(username, email):
    new_user = User(username=username, email=email) ❷
    try:
        db.session.add(new_user) ❸
        db.session.commit() ❹
    except Exception as e:
        db.session.rollback() ❺
    return redirect(url_for('index')) ❻
```

❶ This endpoint requires a username and an email to be passed.

❷ This creates a new User object.

❸ This adds the user to the database without committing the change yet.

❹ This commits the change, only if there were no exceptions in the line before (e.g., username already exists).

❺ If an exception occurred, this code rolls back the change (i.e., undo adding the user).

❻ Regardless of the outcome, this function redirects the browser to the landing page.

READ - Get a specific user

```
@app.route('/read_user/<int:user_id>') ❶
def read_user(user_id):
    user = User.query.get_or_404(user_id) ❷
    return f"User: {user.username}, Email: {user.email}" ❸
```

❶ This endpoint requires a integer user id to be passed.

❷ If the user is not found, this function returns a HTTP error code 404 "Not Found".

❸ If the user is found, this function returns the string similar to the hello-world example, not a full HTML template.

UPDATE - Modify a user

```
@app.route('/update_user/<int:user_id>/<string:username>/<string:email>') ❶
def update_user(user_id, username, email):
    user = User.query.get_or_404(user_id)
    user.username = username ❷
    user.email = email
    try:
        db.session.commit()
    except Exception as e:
        db.session.rollback()
    return redirect(url_for('index'))
```

❶ This endpoint requires the user ID of the entry that the visitor wants to change as well as the new username and email.

❷ Here the values are updated in the User object without committing the changes yet.

DELETE - Remove a user

```
@app.route('/delete_user/<int:user_id>')
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
    try:
        db.session.delete(user)
        db.session.commit()
    except Exception as e:
        db.session.rollback()
    return redirect(url_for('index'))
```

Putting it all together

With the code snippets above you can now create a server that lets clients manipulate the database. The complete `app.py` will then look as follows.

Note the required `redirect`, `url_for`, and `flash` imports from the flask package in line 1

- `redirect` lets you send the client to another route (e.g., to the user profile page after adding a new user).
- `url_for` gives you the url for a route (e.g., index); you have used these in the previous section already in the HTML templates.
- `flash` is used to send pop-up notifications from the server to the client.

app.py

```
from flask import Flask, render_template, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()
```

app.py (continued)

```
@app.route('/')
def index():
    users = User.query.all()
    return render_template('home.html', users=users)

@app.route('/add_user/<string:username>/<string:email>')
def add_user(username, email):
    new_user = User(username=username, email=email)
    try:
        db.session.add(new_user)
        db.session.commit()
        flash(f'User {username} added successfully!', 'success')
    except Exception as e:
        db.session.rollback()
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))

@app.route('/read_user/<int:user_id>')
def read_user(user_id):
    user = User.query.get_or_404(user_id)
    return f"User: {user.username}, Email: {user.email}"

@app.route('/update_user/<int:user_id>/<string:username>/<string:email>')
def update_user(user_id, username, email):
    user = User.query.get_or_404(user_id)
    user.username = username
    user.email = email
    try:
        db.session.commit()
        flash(f'User {username} updated successfully!', 'success')
    except Exception as e:
        db.session.rollback()
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))

@app.route('/delete_user/<int:user_id>')
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
    try:
        db.session.delete(user)
        db.session.commit()
        flash(f'User deleted successfully!', 'success')
    except Exception as e:
        db.session.rollback()
        flash(f'Error: {str(e)}', 'error')
    return redirect(url_for('index'))
```

You can add a user by typing directing your browser to the URL http://127.0.0.1:5000/add_user/jane/jane@asu.edu. It will run the `add_user` function on the server

and redirect you to the homepage. When the homepage is loaded, it will get all the users and you should see the new entry in the list.

You should put nicer error messages into production

The code above injects the raw python error message into the notification that is sent to the user. You should write something that is more helpful to the user.

Next steps to try it out!

- Try out the update and delete endpoints as well.
- Check your manipulations in the MySQL Workbench.
- Try to cause an error message to show up.
- Add Bootstrap and custom styling to the flash notifications.